Module 1

Introduction

Dr. Ngangbam Indrason
Assistant Professor - SCOPE
VIT-AP University, Amaravati

Contents

- What is Operating System?
- What operating systems do
- Computer System Organization
- Operating system operations
  - Process Management, Memory Management, Storage Management, I/O Systems, Protection & Security,
- Types of Operating Systems
  - Batch O.S., Multi programmed O.S., Time Sharing O.S., Distributed O.S., Virtualization, Real Time Embedded Systems, Open source O.S.
- System calls & Overview on Types of System calls.

What is Operating System?

- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner

What is Operating System?

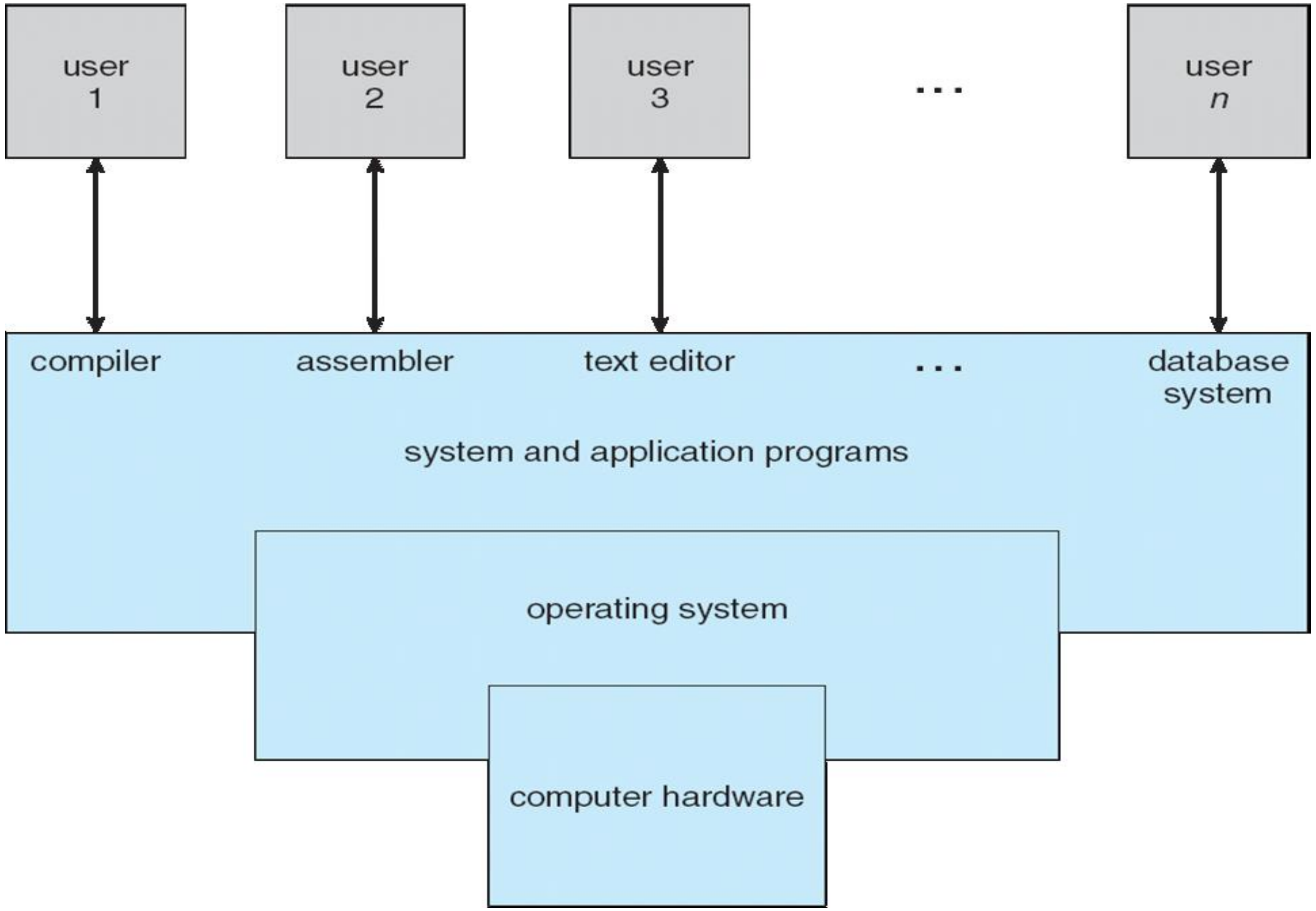
- OS is a **resource allocator**
  - Manages all resources
  - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
  - Controls execution of programs to prevent errors and improper use of the computer
- "The one program running at all times on the computer" is the **kernel**.
- Everything else is either a system program or an application program.

Computer System Structure

- Computer system can be divided into four components:
  - Hardware – provides basic computing resources
    - CPU, memory, I/O devices
  - Operating system
    - Controls and coordinates use of hardware among various applications and users
  - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
    - Word processors, compilers, web browsers, database systems, video games
  - Users

    People, machines, other computers

Four Components of a Computer System



What Operating Systems Do

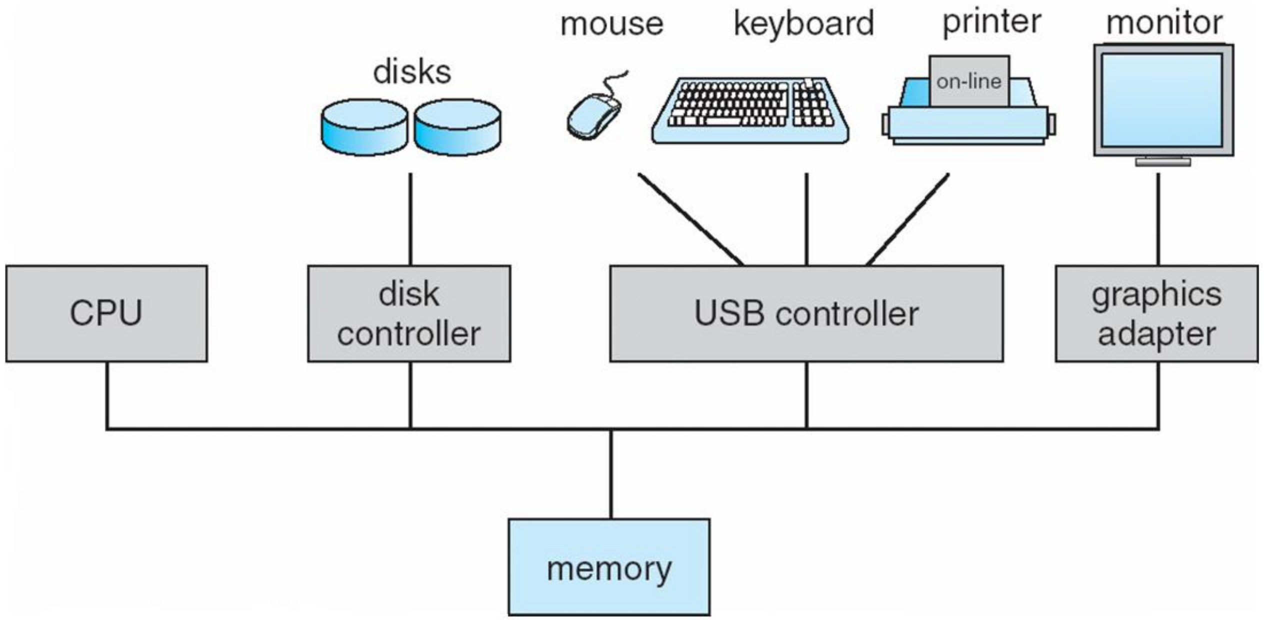
- Controls the hardware and coordinates its use among the various application programs for the various users
- Provides means for proper use of resources in the operation of computer system
- OS is like government; it simply provides an environment within which other programs can do useful work

What Operating Systems Do

- Key Roles:
  - Acts as an intermediary between users and hardware.
  - Manages resources like CPU, memory, storage, and I/O devices.
  - Provides a user interface (CLI or GUI).
- Functions:
  - Process and memory management
  - File and storage management
  - Device and I/O control
  - Security and protection
  - Networking and communication

Computer System Organization

- Computer-system operation
  - One or more CPUs, device controllers connect through common bus providing access to shared memory
  - Concurrent execution of CPUs and devices competing for memory cycles



Computer-System Operations

- I/O devices and the CPU can execute concurrently
- Each device controller is in charge of a particular device type
- Each device controller has a local buffer
- CPU moves data from/to main memory to/from local buffers
- I/O is from the device to local buffer of controller
- Device controller informs CPU that it has finished its operation by causing an interrupt

Process Management

- A process is a program in execution. It is a unit of work within the system. Program is a ***passive entity*** process is an ***active entity***.
- Process needs resources to accomplish its task
  - CPU, memory, I/O, files
  - Initialization data
- Process termination requires reclaim of any reusable resources
- Single-threaded process has one **program counter** specifying location of next instruction to execute
  - Process executes instructions sequentially, one at a time, until completion

Process Management

- Multi-threaded process has one program counter per thread
- Typically, system has many processes, some user, some operating system running concurrently on one or more CPUs
  - Concurrency by multiplexing the CPUs among the processes / threads

Process Management Activities

- The operating system is responsible for the following activities in connection with process management:
  - Creating and deleting both user and system processes
  - Suspending and resuming processes
  - Providing mechanisms for process synchronization
  - Providing mechanisms for process communication
  - Providing mechanisms for deadlock handling

Memory Management

- To execute a program all (or part) of the instructions must be in memory
- All (or part) of the data that is needed by the program must be in memory.
- Memory management determines what is in memory and when to optimize CPU utilization and computer response to users
- Memory management activities
  - Keeping track of which parts of memory are currently being used and by whom
  - Deciding which processes (or parts thereof) and data to move into and out of memory
  - Allocating and deallocating memory space as needed

Storage Management

- Storage Management refers to the way an operating system manages various data storage devices and the data stored on them.
- It includes managing both primary (RAM) and secondary storage (hard drives, SSDs).
- File-System management
  – Files usually organized into directories
  – OS activities include
    - Creating and deleting files and directories
    - Primitives to manipulate files and directories
    - Mapping files onto secondary storage
    - Backup files onto stable (non-volatile) storage media

Performance of Various Levels of Storage

Level	1	2	3	4	5
Name	registers	cache	main memory	solid state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25 - 0.5	0.5 - 25	80 - 250	25,000 - 50,000	5,000,000
Bandwidth (MB/sec)	20,000 - 100,000	5,000 - 10,000	1,000 - 5,000	500	20 - 150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

- Movement between levels of storage hierarchy can be explicit or implicit

I/O System

- One purpose of OS is to hide unique behaviours and complexities of underlying hardware devices from the user.
- I/O subsystem responsible for
  - Memory management of I/O including buffering (storing data temporarily while it is being transferred), caching (storing parts of data in faster storage for performance), spooling (the overlapping of output of one job with input of other jobs)
  - General device-driver interface
  - Drivers for specific hardware devices

Protection and Security

- **Protection** – any mechanism for controlling access of processes or users to resources defined by the OS.
- **Security** – defence of the system against internal and external attacks including denial-of-service, worms, viruses, identity theft, theft of service.
- Systems generally first distinguish among users, to determine who can do what
  - User identities (user IDs, security IDs) include name and associated number, one per user
  - User ID then associated with all files, processes of that user to determine access control
  - Group identifier (group ID) allows set of users to be defined and controls managed, then also associated with each process, file
  - Privilege escalation allows user to change to effective ID with more rights

Types of Operating Systems

- Batch O.S.
- Multi programmed O.S.
- Time Sharing O.S.
- Distributed O.S.
- Virtualization
- Real Time Embedded Systems
- Open source O.S.

Batch Operating System

- Batch OS executes batches of jobs without user interaction. Jobs are grouped and processed sequentially
- **Key Features**: No direct user interaction during execution; Jobs submitted via punch cards or scripts; Uses job scheduling to optimize CPU usage
- **Advantages**: Efficient for large repetitive tasks; Reduces idle CPU time
- **Disadvantages**: No real-time feedback; Debugging is difficult
- **Examples**: IBM OS/360, early UNIX systems

Multi programmed Operating System

- It allows multiple programs to reside in memory and share CPU time. The OS switches between jobs to maximize CPU utilization.
- **Key Features**: Multiple jobs in memory; CPU switches to another job when one waits for I/O; Requires memory management and CPU scheduling
- **Advantages**: Better resource utilization; Increased throughput
- **Disadvantages**: Complex scheduling; Requires sophisticated memory management
- **Examples**: UNIX, Windows NT

Time Sharing Operating System

- It extends multiprogramming by allowing multiple users to interact with the system simultaneously. Each user gets a time slice of the CPU.
- **Key Features**: Interactive user sessions; Uses time quantum for CPU allocation; Supports multiple terminals
- **Advantages**: Fast response time; Efficient for multi-user environments
- **Disadvantages**: Requires complex scheduling; Security and data integrity concerns
- **Examples**: MULTICS, UNIX, Windows Server

Distributed Operating System

- It manages a group of independent computers and makes them appear as a single system to users.
- **Key Features:** Resource sharing across networked systems; Transparency in access and location; Fault tolerance and scalability
- **Advantages:** High reliability and performance; Load balancing across nodes
- **Disadvantages:** Complex synchronization and communication; Security challenges
- **Examples**: Amoeba, Plan 9, Microsoft Azure OS

Virtualization-Based Operating System

- It allows multiple virtual machines (VMs) to run on a single physical machine using a hypervisor.
- **Key Features**: Hardware abstraction; Isolation between VMs; Supports dynamic resource allocation
- **Advantages**: Efficient use of hardware; Easy testing and deployment
- **Disadvantages**: Overhead due to virtualization layer; Performance may degrade under heavy load
- **Examples**: VMware ESXi, Microsoft Hyper-V, KVM

Real-Time Embedded Operating System

- It is designed for systems that require immediate response to external events. Common in embedded systems.
- **Key Features**: Deterministic response time; Priority-based scheduling; Minimal latency
- **Types**:
  - **Hard Real-Time**: Missing deadlines leads to failure
  - **Soft Real-Time**: Occasional deadline misses are tolerable
- **Advantages**: Predictable behaviour; Optimized for specific hardware
- **Disadvantages**: Limited flexibility; Complex development
- **Examples**: VxWorks, RTEMS, FreeRTOS

Open Source Operating System

- OS whose source code is freely available for modification and redistribution.
- **Key Features**: Community-driven development; Transparent architecture; Customizable and extensible
- **Advantages**: Cost-effective; High security through peer review
- **Disadvantages**: May lack professional support; Compatibility issues with proprietary software
- **Examples**: Linux, FreeBSD, ReactOS

System calls

- A system call is a programmatic way for a user-level process to request services from the operating system's kernel.
- It acts as an interface between user applications and the OS, allowing access to hardware and system resources.
- It triggers a switch from user mode to kernel mode for privileged operations.
- They are used for accessing hardware resources, file operations, process control, communication, etc.

Types of System calls

- Process control
  - create process, terminate process
  - end, abort
  - load, execute
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory
  - Dump memory if error
  - Debugger for determining bugs, single step execution
  - Locks for managing access to shared data between processes

Types of System calls

- File management
  - create file, delete file
  - open, close file
  - read, write, reposition
  - get and set file attributes
- Device management
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices

Types of System calls

- Information maintenance
  - get time or date, set time or date
  - get system data, set system data
  - get and set process, file, or device attributes
- Communications
  - create, delete communication connection
  - send, receive messages if message passing model to host name or process name
    - From client to server
  - Shared-memory model create and gain access to memory regions
  - transfer status information
  - attach and detach remote devices

Types of System calls

- Protection
  - Control access to resources
  - Get and set permissions
  - Allow and deny user access

End of Module 1